

第5章 序列与集合算法

5.1 引言

- 序列：考虑元素顺序，元素可重复
集合：不考虑元素顺序，元素不可重复

5.2 二分搜索

5.2.1 经典二分搜索

- 经典二分：通过待查元素与有序序列中间元素比较，将搜索范围减半
时间复杂度 $O(\log n)$

5.2.2 循环序列的二分搜索

- 问题： $x_1, x_2, x_3, \dots, x_n$ 中 $\exists x_i : x_i < x_{i+1} < x_{i+2} < \dots < x_n < x_1 < x_2 < x_3 < \dots < x_{i-1}$ ，找出 x_i 的位置
- 思想：选择合适的元素进行比较，每次排除范围元素，达到 $O(\log n)$ 的复杂度
任取 $x_k, x_l (k < l)$
 - 若 $x_k < x_m$ ，则 $i \notin (k, m]$
 - 若 $x_k > x_m$ ，则 $i \in (k, m]$

5.2.3 二分搜索特殊下标

- 问题：整数序列 $a_1, a_2, a_3, \dots, a_n$ 递增，求是否存在 i 使得 $a_i = i$
- 思想：考查 $a_{n/2}$ ，每次比较可删除一半元素，达到 $O(\log n)$ 的复杂度
 - 若 $a_{n/2} = \frac{n}{2}$ 则直接得解
 - 若 $a_{n/2} < \frac{n}{2}$ 则前半段元素均不满足
 - 若 $a_{n/2} > \frac{n}{2}$ 则后半段元素均不满足

5.2.4 未知长度序列的二分搜索

- 问题：递增序列长度未知，在该序列中查找元素 z
- 思想：控制搜索范围实现 $O(\log n)$ 的查找
 - 若 $z > x_i$ ，则加倍搜索范围，至多 $\log z$ 次
 - 若 $z < x_{2i}$ ，则在 $(i, 2i]$ 范围内搜索即可，至多比较 $\log i$ 次

5.2.5 重叠子序列问题

- **问题：**给定序列 A, B ，确定 i 的最大值使得 B^i 是 A 的子序列
 B^i 表示 B 中每个元素重复 i 次形成的序列，如 $B = b_1b_2b_3$ 则 $B^2 = b_1b_1b_2b_2b_3b_3$
- **思想：**控制 i 的范围实现 $O(\log k)$ 的查找
设 $|A| = m, |B| = n$ 则 $i < \frac{n}{m}$ ，否则 B^i 的长度大于 A 的长度
可在 1 到 $\frac{n}{m}$ 中进行二分搜索，时间复杂度 $O((m+n) \log(m+n))$

5.2.6 方程求解

- **问题：** $f(a)f(b) < 0, f \in C[a, b]$ ，求 $f(x)$ 的零点
- **思想：**二分法逼近函数零点
若 $f(a + \frac{b-a}{2}) = 0$ 则零点就是 $a + \frac{b-a}{2}$
否则可将范围缩小至 $(a, a + \frac{b-a}{2})$ 或 $(a + \frac{b-a}{2}, b)$
 k 步之后可将零点范围缩小至 $\frac{b-a}{2^k}$

5.3 内插搜索

- **思想：**根据待检索值在检索区内的相对位置，在一定的邻域内进行搜索
- **适用情况：**适用于数据分布均匀的序列
- **优点：**可将复杂度由 $O(\log n)$ 降低至 $O(\log \log n)$
- **缺点：**复杂度即使有所优化也程度有限；内插搜索的运算更加复杂

5.4 排序

5.4.1 基于数值的排序

- **计数排序：**直接计算小于每个元素的元素个数，直接将元素放在最终位置上，时间复杂度 $O(n)$
也可将一定数值范围内的元素视为整体，计算小于该整体的元素数目，将元素置于最终的位置范围上，然后对该范围内的元素进行排序
- **桶排序：**对于取值范围为 1 到 m 的 n 个元素，可分配 m 个桶，根据元素的取值放入桶中，最后遍历桶并收集元素，时间复杂度 $\Theta(m+n)$
- **桶排序扩展：**每个桶中放置多个元素，先对每个桶中的元素进行排序，再遍历桶并收集元素
若桶的个数与元素个数相近，则平均时间复杂度 $\Theta(n)$ ，证明如下：
设 n_i 为第 i 个桶中的元素个数，则 $n_i \sim B(n, \frac{1}{n})$ ，且对每个桶中的元素排序的时间为 $E(n_i^2)$
(说明：由于平均每个桶中的元素较少，常系数的影响较大，因此对每个桶中的元素进行排序时，一般不采用 $O(n \log n)$ 的排序算法)
有 $E(n_i) = 1 \quad D(n_i) = 1 - \frac{1}{n_i}$
 $\therefore E(n_i^2) = D(n_i) + [E(n_i)]^2 = 2 - \frac{1}{n} = \Theta(1)$
由于共 n 个桶，所以平均时间为 $\Theta(n)$
- **基数排序：**数据位数相同时，可直接按照字典序进行排序

5.4.2 基于比较的排序

- **插入排序**: 通过扫描已完成排序的 $n - 1$ 个数确定第 n 个数应插入的位置, 时间复杂度 $O(n^2)$
- **选择排序**: 依次从未排序的部分选择最小元素放在其最终位置, 时间复杂度 $O(n^2)$
- **归并排序**: 将序列分为 2 部分, 依次递归地归并排序, 最后合并为有序序列, 时间复杂度 $O(n \log n)$
- **快速排序**: 取任意元素作为主元 (一般取中间元素), 将序列分为 2 个部分, 一部分元素小于主元, 另一部分元素大于主元, 并递归地进行快速排序, 时间复杂度 $O(n \log n)$
- **堆排序**: 建堆, 依次入堆并出队, 堆操作 $O(\log n)$, 总时间复杂度 $O(n \log n)$
- **优化思路**: 按归纳方向分为**插入排序**和**选择排序**
插入排序基础上, 将元素与序列的归并, 优化为序列与序列的归并, 产生**归并排序**; 在此基础上强调分治重点, 产生**快速排序**
选择排序基础上, 优化选择方法, 应用历史的比较信息, 产生**堆排序**
- **决策树原理**: n 个元素的排列数为 $n!$, 排序结果相当于二叉树的叶子结点, 所以二叉树高度至少为 $\log(n!) = \Theta(n \log n)$
基于比较的排序实质上是沿着二叉树向下前进, 每次比较在结点的左右子结点中选择, 直到到达叶子结点, 所以基于比较的排序的时间均为 $\Omega(n \log n)$

5.5 顺序统计问题

5.5.1 最大数最小数问题

- **问题**: 求 $x_1, x_2, x_3, \dots, x_n$ 的最大数与最小数
- **直接选择**: 进行选择排序的单次操作, 依次比较并选择最大数与最小数, 时间约为 $T(n) = 2n$
- **选择优化**: 从 $n - 2$ 扩展到 n 时, 比较 x_{n-1} 与 x_n , 之后较大数与前 $n - 2$ 个数中的最大数比较, 较小数与前 $n - 2$ 个数中最小数比较
递推复杂度 $T(n) = T(n - 2) + 3$ $T(2) = 1$ $T(3) = 3$ $\therefore T(n) = \frac{3n}{2}$, 较直接选择优化了约 25%
- **分治方法**: 分为 $\frac{n}{2}$ 的 2 部分, 分别递归处理得到 M_1, m_1, M_2, m_2 , 之后只要比较 M_1 和 M_2 以及 m_1 和 m_2 即可
递推复杂度 $T(n) = 2T(\frac{n}{2}) + 2$ $\therefore T(n) \approx \frac{3n}{2}$

5.5.2 第 k 小数问题

- **问题**: 求 $x_1, x_2, x_3, \dots, x_n$ 的第 k 小数
- **直接法**: 求 k 次最小数, 时间复杂度 $\Theta(kn)$
- **快排优化**: 选择主元将集合分为 2 部分, 比较主元下标 $index$ 和 k 的相对位置
 - 若主元位置 $index$ 恰好为 k 则直接得解
 - 若主元位置 $index$ 小于 k 则 $k \leftarrow k - index$ 之后在后一半搜索
 - 若主元位置 $index$ 大于 k 则在前一半搜索

◦ 时间复杂度 $\Theta(n)$

证明：

$$\begin{aligned}
 T(n) &= (n-1) + \frac{1}{n} \left[\sum_{i=1}^{k-1} T(n-i) + \sum_{i=k+1}^n T(i-1) \right] \\
 &= (n-1) + \frac{1}{n} \left[\sum_{i=n-k+1}^{n-1} T(i) + \sum_{i=k}^{k-1} T(i) \right] \\
 &\leq (n-1) + \frac{1}{n} \max_k \left[\sum_{i=n-k+1}^{n-1} T(i) + \sum_{i=k}^{k-1} T(i) \right]
 \end{aligned}$$

取 $k = \lceil \frac{n}{2} \rceil$ 时最大，设 $i < n$ 时 $T(i) < 4i$ ，则有

$$\begin{aligned}
 T(n) &= (n-1) + \frac{1}{n} \left[\sum_{i=n-\lceil \frac{n}{2} \rceil+1}^{n-1} 4i + \sum_{i=\lceil \frac{n}{2} \rceil}^{n-1} 4i \right] \\
 &\leq (n-1) + \frac{4}{n} \left[\sum_{i=\lceil \frac{n}{2} \rceil}^{n-1} i + \sum_{i=\lceil \frac{n}{2} \rceil}^{n-1} i \right] \\
 &= (n-1) + \frac{8}{n} \sum_{i=\lceil \frac{n}{2} \rceil}^{n-1} i \\
 &\leq (n-1) + \frac{8}{n} \left(\frac{n(n-1)}{2} - \frac{\frac{n}{2}(\frac{n}{2}-1)}{2} \right) \\
 &= 4n - 3 < 4n
 \end{aligned}$$

$\therefore T(n) = \Theta(n)$

5.6 串匹配

- **问题：**已知字符串 A, B ，求 B 在 A 中出现的位置

5.6.1 BF算法

- **思想：**从 A 中出现 $B[1]$ 的位置开始匹配，若遇到不能匹配的字符则重新匹配，最坏情况下的时间复杂度 $O(mn)$

5.6.2 KMP算法

- **思想：**利用匹配过程中的历史信息，设计不回溯算法
- **$next[]$ 的思想与实现：** $next[i]$ 为使得 $B[i-j..i-1] = B[1..j]$ 最大的 j

◦ 构建原理：由 $next[1..i-1]$ 强归纳得到 $next[i]$

最简单情况 $B[i-1] = B[next[i-1]+1]$ ，此时 $next[i] = next[i-1] + 1$

若 $B[i-1] \neq B[next[i-1]+1]$

- 由 $next[]$ 的性质可知， $B[i-1]$ 之前的 $next[i-1]$ 个元素与 B 的前 $next[i-1]$ 个元素相同

即 $B[i - next[i-1] - 1..i-1]$ 与 $B[1..next[i-1]]$ 相同

- 关注 $B[next[i-1]+1]$ ，由 $next[]$ 的性质可知， $B[next[i-1]+1]$ 之前的 $next[next[i-1]+1]$ 个元素与前 $next[next[i-1]+1]$ 个元素相同

即 $B[next[i-1] - next[next[i-1]+1]..next[i-1]]$ 与 $B[1..next[next[i-1]+1]]$ 相同

- 可以比较 $B[next[next[i-1]+1]+1]$ 与 $B[i-1]$

若匹配则 $next[i] = next[next[i-1]+1] + 1$

若不匹配则进行 $j = next[j] + 1$ 之后再次匹配

直到匹配成功，此时 $next[i] = j$ ；或 $j = 0$ ，此时 $next[i] = 0$

```
// Algorithm: next_in_KMP
// Input: string key
// Output: int[] next
int next[key.length];
next[1] = -1, next[2] = 0;
for (int i=3 to key.length) begin
    int j = next[i-1]+1;
    while key[i-1] != key[j] && j>0 begin
        j = next[j]+1;
    endwhile
    next[i] = j;
endfor
return next[];
```

- **KMP算法的构建：**模式串与目标串出现字符不匹配时，将模式串向右滑动，使得 $B[i - next[i] - 1..i-1]$ 与 $B[1..next[i]]$ 对齐，比较 $B[next[i]+1]$ 与 $B[i]$ 即可

```
// Algorithm: KMP
// Input: string obj, string key
// Output: int index
int j = 0;
int[] next = next_in_KMP(key);
for (int i=1 to obj.length) begin
    while (j > 0 && key[j+1] != obj[i]) j = next[j+1];
    if (key[j+1] == obj[i]) j++;
    if (j==key.length) return i-j;
endfor
return 0;
```

5.6.3 BM算法

- **思想**：在利用模式串特性的基础上，再对失配的目标串字符进行利用
- **坏字符规则**：模式串与目标串不匹配时，记目标串字符为坏字符
出现坏字符时，模式串向右滑动，直到目标串的坏字符与模式串的相同字符对齐（若坏字符在模式串未出现，则使模式串首位与坏字符的右邻位对齐）
可建立二维数组，表示失配时，根据坏字符，模式串应滑动到的位置
- **好后缀规则**：模式串与目标串已匹配的后缀，记这部分字符串为好后缀
出现失配时，模式串向右滑动，直到模式串中再次出现的好后缀与目标串相同字符串对齐（若好后缀在模式串中完全未出现，则直接向右滑动到目标串中好后缀的右邻位；若好后缀的部分后缀子串与模式串的前缀子串重合，则将模式串移动到和前缀子串重合的位置）
好后缀规则与KMP算法有部分相似之处

5.7 中位数问题

5.7.1 加权中位数

- **问题**：给定数组 $X[1..n]$ ，以及 $W[1..n]$ 表示各元素权值， $\sum_i w_i = 1$ 且 $w_i \neq 0$ ，加权中位数 x_k 满足 $\sum_{x_i < x_k} w_i < \frac{1}{2}$ ， $\sum_{x_i > x_k} w_i > \frac{1}{2}$
- **直接方法**：排序后，从小到大遍历元素，统计 $\sum_{x_i < x_k} w_i$ ，返回首次 $\sum_{x_i < x_k} w_i > \frac{1}{2}$ 的前个位置，时间复杂度 $\Theta(n \log n)$
- **快排优化**：选择主元将集合分为 2 部分，比较前半权值、后半权值和 $\frac{1}{2}$ 的大小，若有一部分大于 $\frac{1}{2}$ ，则在相应部分递归求解（需要修正权值及阈值）

5.7.2 2个等长有序数组的中位数

- **问题**：给定增序数组 $A[1..n]$, $B[1..n]$ ，求这 $2n$ 个数的中位数
- **直接方法**：归并排序的归并过程，依次出队 n 次即可
- **二分优化**：比较 $A\left[\left\lceil \frac{n}{2} \right\rceil\right]$ 和 $B\left[\left\lceil \frac{n}{2} \right\rceil\right]$
 - 若 $A\left[\left\lceil \frac{n}{2} \right\rceil\right] = B\left[\left\lceil \frac{n}{2} \right\rceil\right]$ ，则均是中位数
 - 若 $A\left[\left\lceil \frac{n}{2} \right\rceil\right] < B\left[\left\lceil \frac{n}{2} \right\rceil\right]$ ，则排除 $A\left[\left\lceil \frac{n}{2} \right\rceil + 1..n\right]$ 和 $B\left[1..\left\lceil \frac{n}{2} \right\rceil\right]$ 后递归求解
 - 若 $A\left[\left\lceil \frac{n}{2} \right\rceil\right] > B\left[\left\lceil \frac{n}{2} \right\rceil\right]$ ，则排除 $A\left[1..\left\lceil \frac{n}{2} \right\rceil\right]$ 和 $B\left[\left\lceil \frac{n}{2} \right\rceil + 1..n\right]$ 后递归求解
 - 时间复杂度 $O(\log n)$

5.8 回文串

- **问题**：找出目标串中尽可能长的回文串
- **直接方法**：暴力搜索，时间复杂度 $O(n^2)$
- **Manacher算法**：可求出奇数长度的回文串
 - 归纳假设：
 - 已找到分别以前 $i - 1$ 个元素为中心的最长回文串半径，记为 $d[i]$

- 已找到的回文串中，右边界最靠右的回文串，其右边界记为 R ，则有 $R \geq i - 1$ （因为至少 $obj[i - 1]$ 本身是回文串）
- $i > R$ ，则暴力搜索求解 $d[i]$ ，并更新 R
 - $i \leq R$ ，则 $\exists j, obj[i] = obj[j]$ ，且在某一回文串中，关于中心位置对称
 - $i + d[j] < R$ ，表示以 i 为中心的最长回文串，其右边界不超过 R ，所以 $d[i] = d[j]$
 - $i + d[j] \geq R$ ，则可以从 $R + 1$ 位置开始，检验关于 i 对称的位置，因为 $[i..R]$ 范围内的字符以匹配
- 偶数长度的回文串**：如 123321，可将原字符串中相邻的元素之间加入特殊字符如 1@2@3@3@2@1
 此时原字符串中偶数长度的回文串，在新字符串中均为以 @ 为中心的奇数长度的回文串
- 时间复杂度**：由于 R 不会回溯，所以复杂度 $\Theta(n)$

5.9 序列修正问题

- 问题**：记插入 1 个字符或删除 1 个字符或替换 1 个字符为 1 次操作，求 $A[1..n]$ 修改为 $B[1..m]$ 的最少操作数
- 动态规划**：记 $C(i, j)$ 为 $A[1..i]$ 变为 $B[1..j]$ 的最小代价
 - 初始化： $C(i, 0) = i, C(0, j) = j$
 - 递推：

$$d(i, j) = \begin{cases} 0 & a_i = b_j \\ 1 & a_i \neq b_j \end{cases}$$

$$C(i, j) = \begin{cases} C(i - 1, j) + 1 & (delete) \\ C(i, j - 1) + 1 & (add) \\ C(i - 1, j - 1) + d(i, j) & (replace) \end{cases}$$

- 求解： $C(n, m)$
- 时间复杂度：表中每个元素的求解均为 $O(1)$ ，所以总复杂度 $O(mn)$

5.10 频繁元素问题

5.10.1 查找众数

- 问题**：若 x 在序列 $X[1..n]$ 中出现次数超过 $\frac{n}{2}$ ，称其为序列中的众数，求序列的众数或说明其不存在
- 直接方法**：
 - 暴力搜索：统计 $A[1..n]$ 中每个元素的出现次数
 - 中位数查找：用 $O(n)$ 找出中位数，再用 $O(n)$ 找出中位数出现的次数是否大于 $\frac{n}{2}$
- 候选轮替**：删除序列中的 2 个不同的数后，原序列的众数是新序列的众数
 所以可以扫描序列更新候选位置
 - 若候选位置为空，则当前元素进入候选位置并设次数为 1

- 若候选位置与当前元素相同，则次数 +1
- 若候选位置与当前元素不同，则次数 -1；此时若次数为 0，则清空候选位置
- 扫描完成后，众数必存活，非众数也可能存活，需要验证存活者是否为众数
- 时间复杂度 $O(n)$ ，包括最坏情况

5.10.2 频繁元素

- **问题：**在数据中找出出现最频繁的 t 个元素
- **精确算法：**为每种数据取值设立计数器，可在数据规模较小时使用，得到精确的结果
- **近似算法：**牺牲部分正确性，以换取空间复杂度的较大优化
借鉴候选轮替思路，只保留 k 个计数器，扫描全数组
 - 若当前元素有对应计数器，则对应次数 +1
 - 若当前元素没有对应计数器
 - 若有空计数器，则填入当前元素，且将次数设为 1
 - 若没有空计数器，则将所有计数器 -1，并丢弃所有次数为 0 的计数器合当前元素
 - 正确性分析：出现次数 $\geq \frac{n}{k+1}$ 的元素一定会出现在计数器中，但计数器中的元素不一定都出现次数 $\geq \frac{n}{k+1}$ ，还需进一步验证出现次数是否 $\geq \frac{n}{k+1}$
特别地， $k = 1$ 时，即为查找众数的候选轮替思路

5.11 最长单调序列

- **问题：**给定由不同整数组成的序列，求最长递增子列（子列放回原序列时，不必连续，但元素在子列中的相对位置必须与原序列相同）
- **归纳假设1：**已经求出原序列前 m 位的最长递增序列
- **归纳假设2：**已经求出原序列前 m 位中末位数最小的递增子列
- **归纳假设3：**原序列前 m 位中，最长递增子列长度为 k ，已经求出所有长度 $l < k$ 的子序列中，末位数最小的递增子列，记为 $BIS(l)$
根据归纳假设， $BIS(l)$ 关于 l 是递增的（否则更小的数可以添加在更长的序列后面），因此扩充 BIS 数组时可以使用二分法查找
理论复杂度 $O(n \log n)$

5.12 查找前两大元素

- **问题：**给定序列，求序列的最大元素和次大元素
- **直接方法：**2 轮遍历，每次分别找出最大元素， $T(n) = 2n - 3$
- **分治法：**
 - **常规分治：**只需要比较 M_1, M_2 ，若 $M_1 \geq M_2$ 则比较 m_1, M_2 ，否则比较 M_1, m_2
 $T(2) = 1, T(n) = 2T(\frac{n}{2}) + 2 \quad T(n) = \frac{3n}{2} - 2$
 - **间接分治：**将 $A[i]$ 与 $A[i + \frac{n}{2}]$ 对应起来，调整位置使得 $A[i] \leq A[i + \frac{n}{2}]$
在 $A[\frac{n}{2} + 1..n]$ 中找到最大数 $A[\frac{n}{2} + f]$ 即为整个数组最大数，再在 $A[\frac{n}{2} + 1..n]$ 中找到次

大数 $A[\frac{n}{2} + s]$ ，则整个数组的次大数必为 $A[f]$ 和 $A[\frac{n}{2} + s]$ 其中之一

$$T(2) = 1, T(n) = \frac{n}{2} + T(\frac{n}{2}) + 1 \quad T(n) = n + \log n - 2$$

- **候选法**：将元素两两比较，并将结果保留在二叉树中，选出最大数后回溯曾与最大数比较过的元素，在其中选出数组的次大数

5.13 求多重集合的模

- **问题**：定义多重集合为元素可重复的集合，定义多重集合的模为出现频率最高的元素，求给定多重集合的模
- **直接方法**：每遇到元素就遍历数组，并更新最高频元素，复杂度 $O(n^2)$
- **基于排序**：排序后顺序统计，复杂度 $O(n \log n)$
- **快排优化**：将集合分为 $< pivot, pivot, > pivot$ 3 部分，前 2 部分递归求解找最高频数的元素，并与 $pivot$ 的频数比较，复杂度 $O(n \log n)$
- **递归终止优化**：分治前后两部分时，若前后 2 部分的规模已经小于 $pivot$ 的频数，则可提前结束分治，复杂度 $O(n \log \frac{n}{M})$